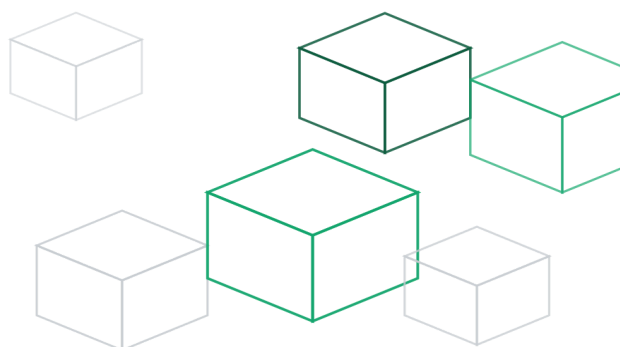




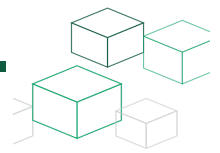
TechMind

Conhecimento Técnico Organizado



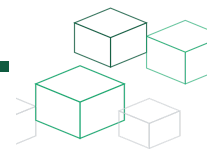
Documentação do Projeto | Backend API

Hackathon ONE | G9



SUMÁRIO

1. O Projeto
2. Docker Local
3. Diagrama de Sequência
4. Endpoints e Funcionalidades



1. O PROJETO

O **TechMind** é uma aplicação desenvolvida para organizar, classificar e recuperar conteúdos técnicos de forma inteligente. A proposta do projeto é resolver um problema comum em ambientes de estudo, equipes de tecnologia e bases internas de conhecimento: a dificuldade de encontrar rapidamente materiais relevantes sobre temas como backend, frontend, cloud, banco de dados, segurança e data science. Em vez de depender apenas de uma busca textual simples ou de organização manual, o TechMind combina backend, inteligência artificial, banco de dados Oracle e recursos de busca semântica para tornar o conhecimento técnico mais acessível.

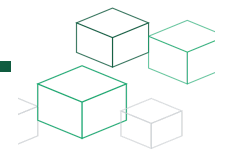
A aplicação funciona como uma biblioteca inteligente. O usuário pode cadastrar conteúdos com título e texto, e o sistema realiza automaticamente a classificação desse material. A partir do conteúdo enviado, o backend aciona um serviço de modelo preditivo em Python, responsável por identificar a categoria mais provável, calcular uma probabilidade de confiança e gerar informações adicionais relacionadas ao texto. Além disso, o sistema também atribui um nível de dificuldade ao conteúdo, como básico, intermediário ou avançado. Dessa forma, cada material cadastrado passa a ter metadados úteis para busca, filtragem e organização.

Um dos diferenciais do projeto é que ele não depende cegamente da classificação automática. O TechMind exibe alertas de baixa confiança quando o modelo não tem segurança suficiente sobre a categoria atribuída. Nesses casos, o usuário pode revisar, confirmar ou corrigir as tags do conteúdo. Essa abordagem cria um fluxo híbrido entre inteligência artificial e revisão humana, tornando o sistema mais confiável para um cenário real de uso.

Além da busca tradicional por tags, o TechMind também possui busca semântica. Isso significa que o usuário não precisa pesquisar exatamente pelas mesmas palavras cadastradas no conteúdo. O sistema utiliza vetores e embeddings armazenados no Oracle Database para comparar significado entre textos e consultas. Com isso, a aplicação consegue encontrar conteúdos relacionados mesmo quando a busca é feita com termos diferentes, mas semanticamente próximos.

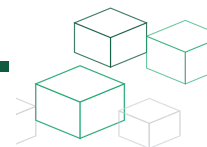
O projeto também inclui funcionalidades voltadas à experiência do usuário, como favoritos, histórico de conteúdos acessados e tags pessoais. As tags pessoais permitem que cada usuário organize conteúdos de acordo com seus próprios critérios, sem alterar a classificação global da biblioteca. Já os favoritos e o histórico ajudam a retomar materiais importantes e acompanhar a navegação dentro da plataforma.

Em termos técnicos, o TechMind é composto por um frontend estático em HTML, CSS e JavaScript, um backend em Java com Spring Boot, um serviço de IA em Python com FastAPI e um banco de dados Oracle. Localmente, esses serviços são orquestrados por Docker Compose. Em cloud, a arquitetura pode ser distribuída entre bucket público para o frontend, Compute Instance para backend e model-service, e Oracle Database gerenciado na OCI. O resultado é um MVP funcional que demonstra classificação automática, organização inteligente de conteúdo, busca semântica e integração entre múltiplos componentes modernos de uma arquitetura cloud-native.

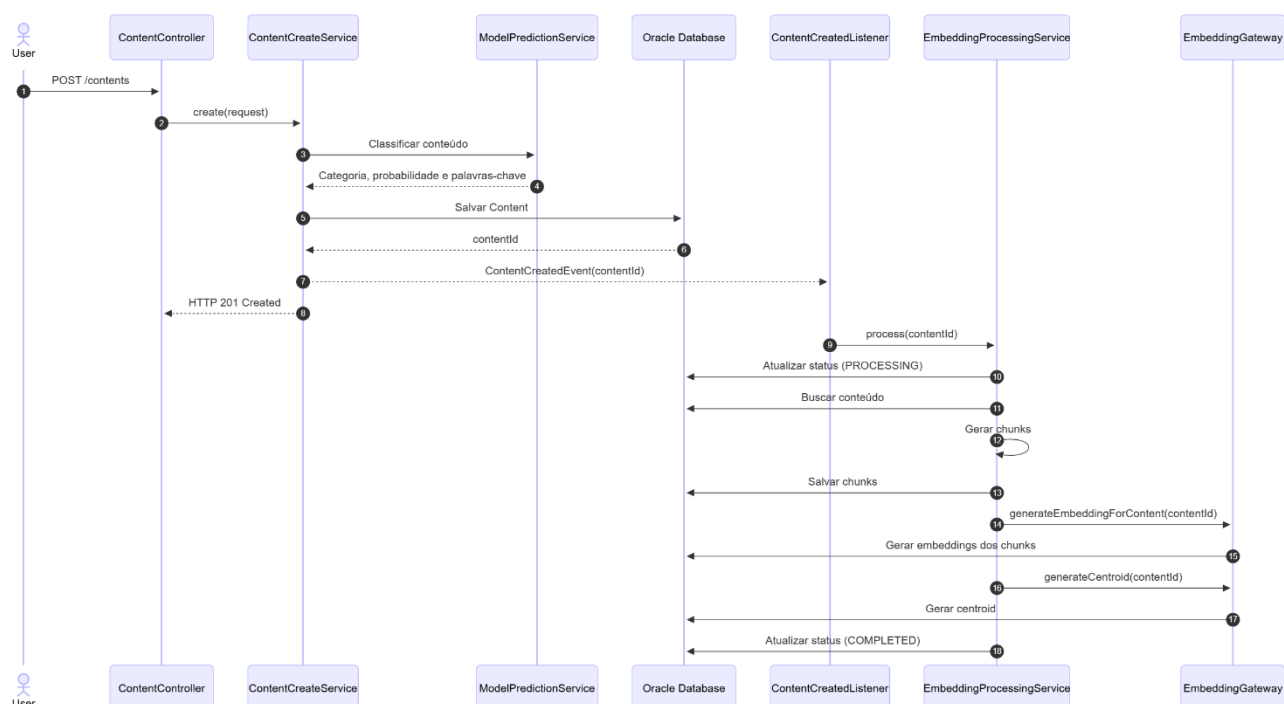


2. DOCKER LOCAL

Para rodar o projeto TechMind localmente, é necessário ter o Docker Desktop e o Git instalados na máquina. Com essas ferramentas disponíveis, o primeiro passo é clonar o repositório do projeto pelo comando `git clone https://github.com/No-Country-simulation/g9-br-techmind-team10.git` e, em seguida, acessar a pasta criada com `cd g9-br-techmind-team10`. Depois disso, é preciso colocar o arquivo do modelo preditivo na pasta correta do projeto, em `model-service/models/modelo_techmind.pkl`. Esse arquivo não deve ser enviado para o GitHub, pois é um artefato local do modelo de machine learning. Também é necessário configurar as variáveis de ambiente do projeto, conforme o arquivo `.env` ou `.env.example`, quando disponível, incluindo dados como credenciais locais, chave JWT, chave de API e modo de execução do serviço de predição. Com tudo configurado, o projeto pode ser iniciado com o comando `docker compose up --build`. Na primeira execução, esse processo pode demorar alguns minutos, porque o Docker precisa baixar as imagens necessárias, instalar dependências do backend Java, instalar dependências do serviço Python, preparar o Oracle Database, aplicar as migrations do banco e construir as imagens da aplicação. Depois que os containers estiverem ativos, o frontend poderá ser acessado em `http://localhost/`, a API do backend em `http://localhost:8080` e a documentação Swagger em `http://localhost:8080/swagger-ui/index.html`. Para verificar se os serviços estão rodando corretamente, pode-se usar o comando `docker compose ps`, e para acompanhar os logs da aplicação, `docker compose logs -f`. Ao cadastrar um conteúdo pelo frontend, o backend chama o serviço Python com o modelo `.pkl`, realiza a classificação do texto, gera tags, salva os dados no banco Oracle e permite que o conteúdo seja pesquisado na biblioteca. Para parar a execução do projeto, utiliza-se `docker compose down`. Caso seja necessário apagar também os dados persistidos localmente, pode-se usar `docker compose down -v`, mas esse comando deve ser usado com cuidado, pois remove os volumes do banco de dados local.



3. DIAGRAMA DE SEQUÊNCIA



4. ENDPOINTS E FUNCIONALIDADES

4.1 CADASTRO DE CONTEÚDO

Objetivo

Permitir que um usuário autenticado envie um novo conteúdo técnico para ser classificado, salvo e disponibilizado na biblioteca.

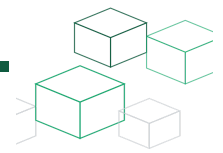
Endpoint

- POST /content

Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.

Autenticação

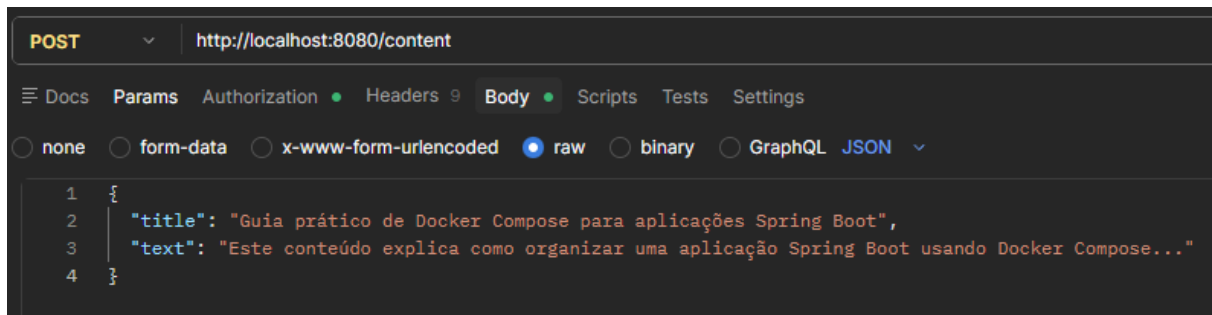


- Bearer Token

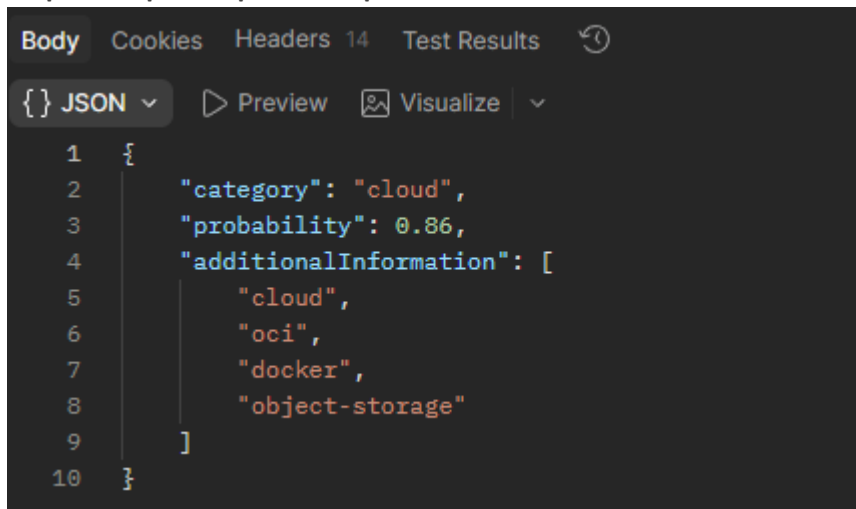
Body da requisição

```
{
  "title": "value",
  "text": "value"
}
```

Exemplo de requisição



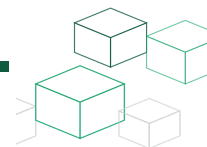
Resposta esperada pelo exemplo



A resposta retorna que o conteúdo foi persistido com sucesso e também informa a categoria e tags que foram associadas a ele.

O que acontece internamente

1. O backend recebe título e texto.
2. O conteúdo passa pelo serviço de classificação.
3. A categoria e as tags são geradas.
4. O conteúdo é salvo no banco de dados.
5. O conteúdo fica disponível para busca e consulta.



4.2 HISTORICO DE ACESSO

Objetivo

- Registrar e disponibilizar o histórico de visualizações e interações do usuário com os conteúdos da plataforma.

Endpoint

- GET /historico

Pré-requisitos

- Backend em execução.
- Banco de dados conectado.
- Usuário autenticado no sistema.

Autenticação

- Bearer Token (JWT).

O que acontece internamente

1. O usuário realiza uma requisição para visualizar os conteúdos.
2. O backend valida o token de autenticação.
3. O sistema registra o ID do usuário e o conteúdo acessado com o carimbo de data/hora (timestamp).
4. A API retorna a listagem ordenada do histórico de acesso para o frontend.

4.3 BUSCA POR TAGS

Objetivo

Permitir que um usuário autenticado consulte conteúdos técnicos previamente cadastrados a partir de uma ou mais tags associadas ao conteúdo.

Endpoint

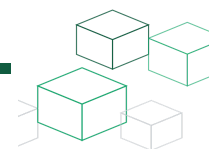
- GET /content/search?tags=value
- O parâmetro tags deve ser enviado na URL da requisição. Ele representa uma ou mais tags que serão utilizadas para filtrar os conteúdos cadastrados.
- Não há body nessa requisição, pois a busca é feita por meio de parâmetros enviados na URL.

Exemplo com uma tag

- GET /content/search?tags=docker

Exemplo com mais de uma tag

- GET /content/search?tags=docker,cloud



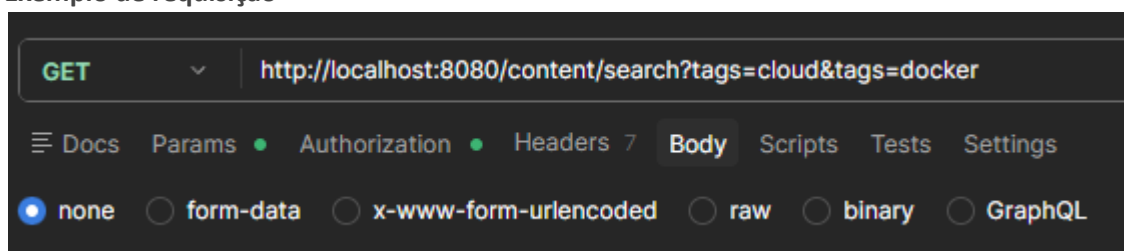
Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdos previamente cadastrados no banco de dados.
- Tags associadas aos conteúdos.

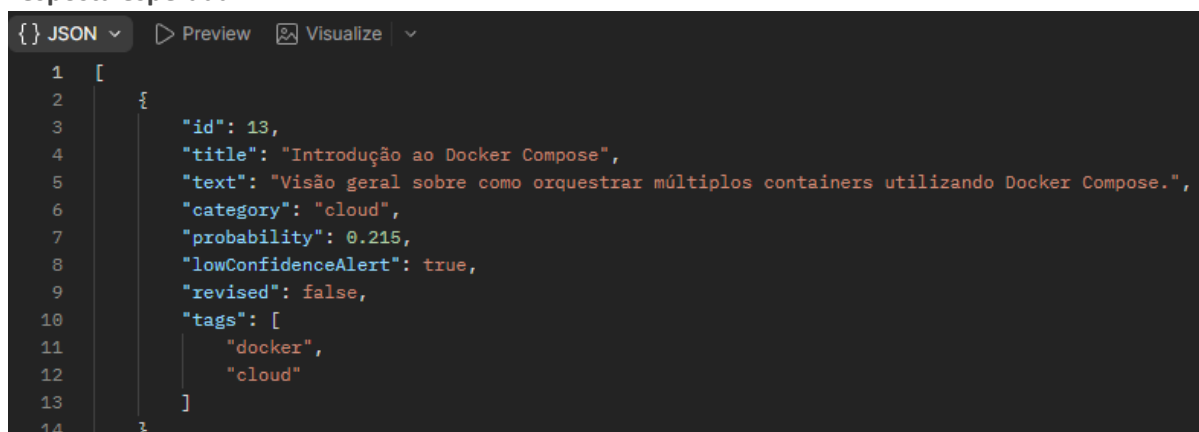
Autenticação

- Bearer Token

Exemplo de requisição



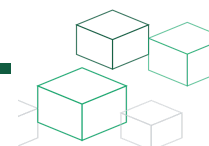
Resposta esperada



A resposta retorna uma lista de conteúdos encontrados a partir das tags informadas. Cada item da lista representa um conteúdo salvo na biblioteca, contendo informações como título, texto, categoria, probabilidade da classificação, status de baixa confiança, status de revisão e tags associadas. Caso nenhum conteúdo seja encontrado para a tag informada, a API retorna uma lista vazia

O que acontece internamente

1. O frontend ou Postman envia uma requisição GET para o endpoint de busca.
2. O backend recebe as tags informadas na URL.
3. O sistema normaliza as tags recebidas.
4. O backend consulta o banco de dados para encontrar conteúdos associados às tags.
5. Os conteúdos encontrados são retornados em formato JSON.
6. O usuário visualiza os resultados na biblioteca.



4.4 SISTEMA DE LOGIN

Objetivo

- Responsável pelo cadastro e autenticação de usuários no sistema. Usa JWT, criptografia de senhas e token de acesso para endpoints bloqueados.

Endpoints

- POST/auth/register
- POST/auth/login

Pré-requisitos

- Backend em execução.
- Banco de dados conectado.

Autenticação

- Não precisa. Esses endpoints não são bloqueados.

Exemplo de requisição do Register

```
POST http://localhost:8080/auth/register

{
  "name": "Guilherme",
  "email": "guilherme@gmail.com",
  "password": "789456"
}
```

Resposta esperada

```
{
  "name": "Guilherme",
  "email": "guilherme@gmail.com"
}
```

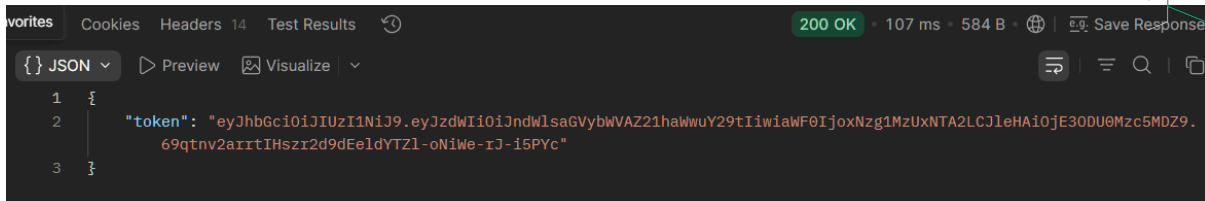
- Vai ser cadastrado o usuário e vai mostrar os dados que foram inseridos no banco, em JSON

Exemplo de requisição do Login

```
POST http://localhost:8080/auth/login

{
  "email": "guilherme@gmail.com",
  "password": "789456"
}
```

Resposta esperada



```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJndWlsaGVybWVhZ21haWwY29tIiwiaWF0IjoxNzg1MzUxNTA2LCJleHAiOiE3ODU0Mzc5MDZ9.69qtnv2arrtIHSzr2d9dEeldYTZl-oNiWe-1J-15PYc"
}
```

- Ao fazer o login e estiver correto, vai ser retornado um token de autenticação que deve ser usado para poder acessar os endpoints bloqueados.

O que acontece internamente

1. Ao receber os dados de cadastro, o backend vai registrar esse usuário no banco de dados, com a senha criptografada
2. Com o usuário cadastrado, ao fazer o login com ele, vai acontecer uma verificação do email e senha para ver se eles existem no banco de dados
3. Se a verificação passar, o token de autenticação vai ser gerado e vai poder ser usado nos endpoints bloqueados.

4.5 CONTADOR DE CONTEÚDO

Objetivo

- Responsável pela contagem de conteúdos dentro de uma categoria específica.

Endpoints

- **GET/content/count**

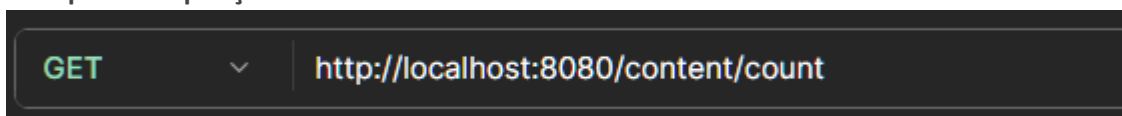
Pré-requisitos

- Backend em execução.
- Banco de dados conectado.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado.
- Conteúdos cadastrados no Banco de Dados.

Autenticação

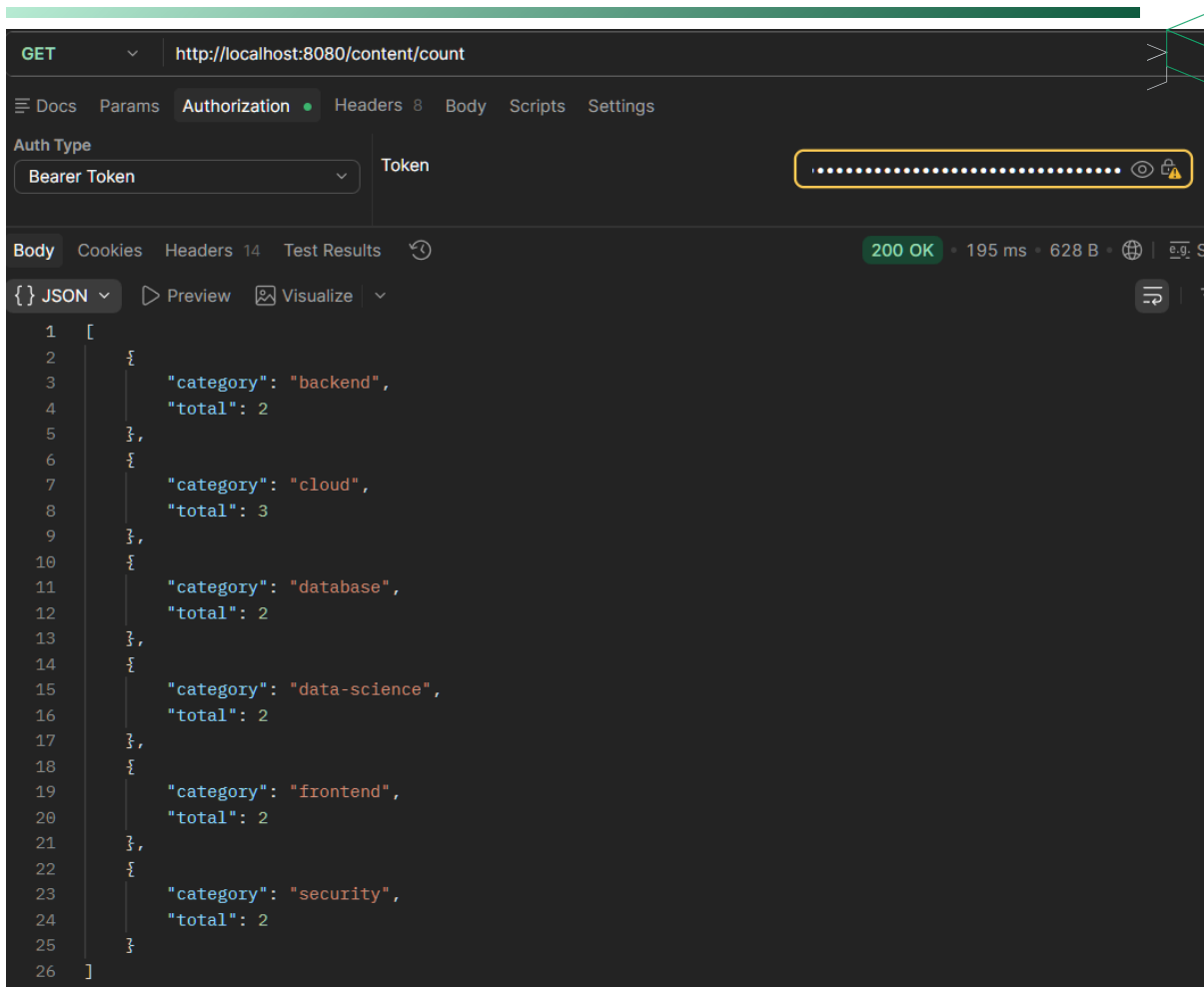
- Bearer Token

Exemplo de requisição



```
GET http://localhost:8080/content/count
```

Resposta esperada



- Vai retornar a categorias e suas contagens de conteúdo

O que acontece internamente

1. O Backend manda uma solicitação para o banco pedindo a contagem dos conteúdos dentro de cada categoria registrada
2. Quando o Banco receber a requisição, ele vai retornar a contagem dos conteúdos em cada categoria.

4.6 CRIAÇÃO E USO DE TAGS PERSONALIZADAS

Objetivo

Permitir que um usuário autenticado crie tags próprias para organizar conteúdos da biblioteca de forma personalizada.

As tags personalizadas são diferentes das tags geradas automaticamente pelo sistema. Elas pertencem ao usuário logado e servem para marcações individuais, como ler depois, favorito, revisar, importante, estudar java, entre outras.

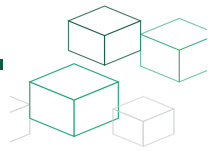
Endpoint para criar tag personalizada

- **POST /content/{contentId}/personal-tags**

Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.

- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdo já cadastrado no sistema.



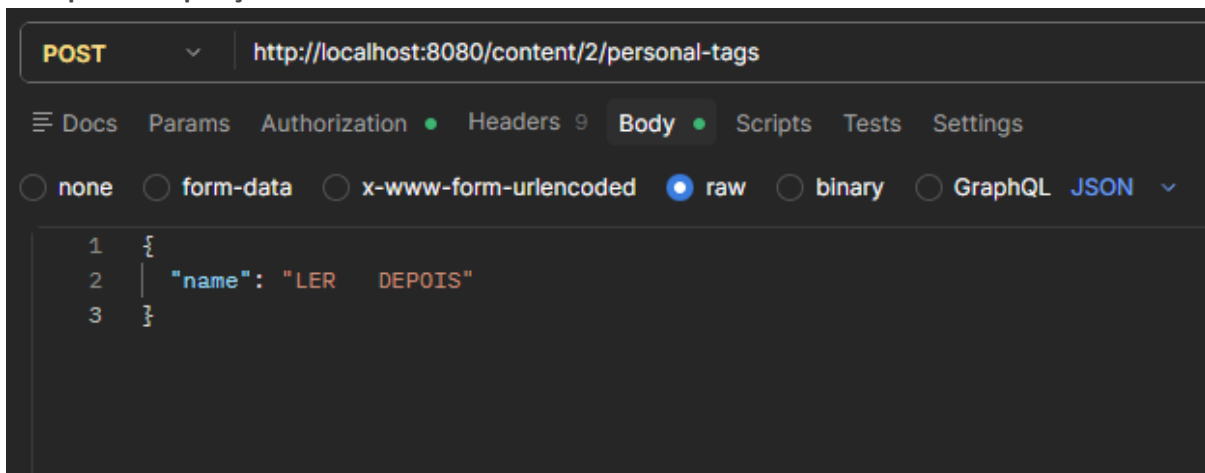
Autenticação

- Bearer Token

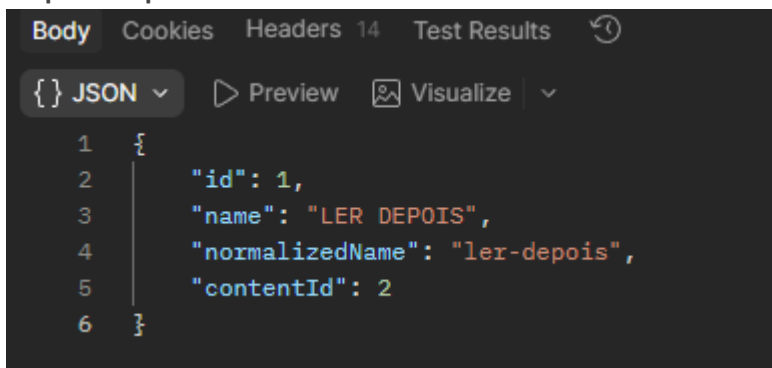
Body da requisição

```
{
  "name": "valor"
}
```

Exemplo de requisição



Resposta esperada



A resposta retorna a tag personalizada criada, informando seu identificador, o nome original informado pelo usuário, o nome normalizado e o conteúdo ao qual ela foi associada.

O campo `normalized_name` é a versão padronizada da tag, usada internamente para facilitar buscas e evitar duplicidades. Por exemplo:

Ler depois → ler-depois

Estudar Java → estudar-java

Conteúdo Prioritário → conteudo-prioritario

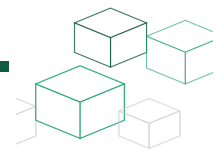
4.7 LISTAGEM DE TAGS PERSONALIZADAS

Objetivo

Permitir que o usuário consulte todas as tags personalizadas que ele adicionou a um conteúdo específico.

Endpoint

- GET `/content/{contentId}/personal-tags`
- Essa requisição não possui body.



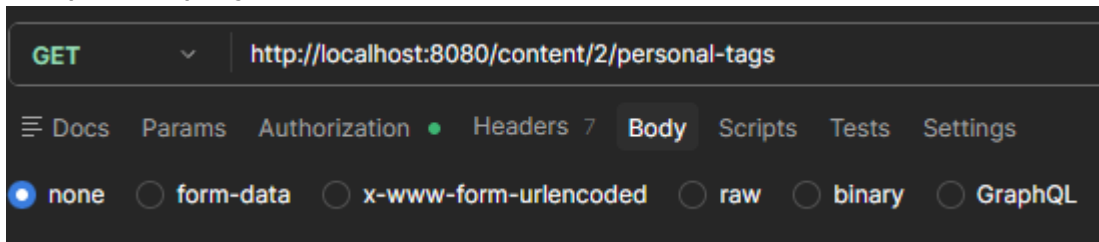
Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdo já cadastrado no sistema.
- Tag personalizada previamente criada

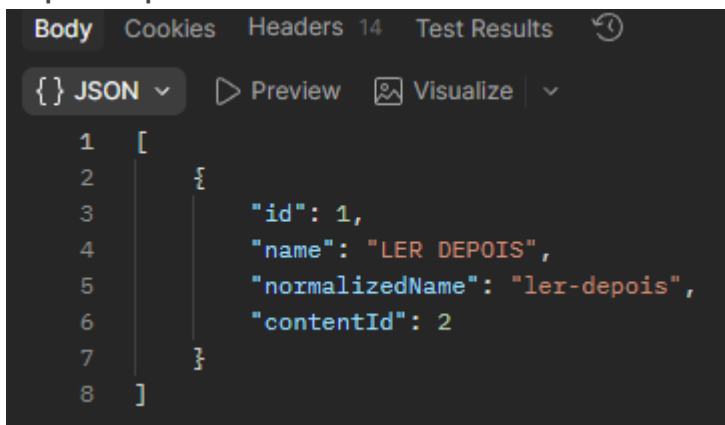
Autenticação

- Bearer Token

Exemplo de requisição



Resposta esperada



Caso o conteúdo não tenha tags personalizadas para aquele usuário, a API retorna uma lista vazia.

4.8 BUSCA POR TAG PERSONALIZADAS

Objetivo

Permitir que o usuário encontre conteúdos da biblioteca a partir das tags personalizadas criadas por ele. Diferente da busca por tags automáticas, essa busca considera apenas as tags pessoais do usuário autenticado.

Endpoint

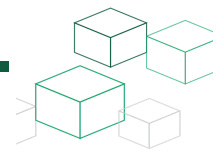
- GET /content/personal-tags/search?tags=value

Exemplo com uma tag

- GET /content/personal-tags/search?tags=ler-depois

Exemplo com mais de uma tag

- GET /content/personal-tags/search?tags=ler-depois,importante



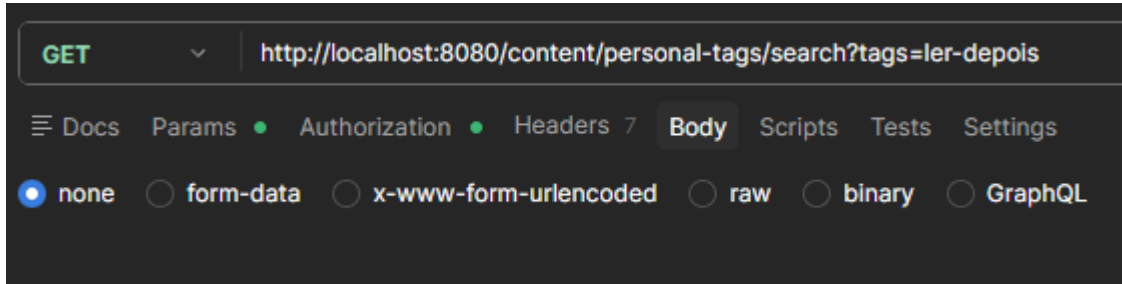
Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdo já cadastrado no sistema.
- Tag personalizada previamente criada

Autenticação

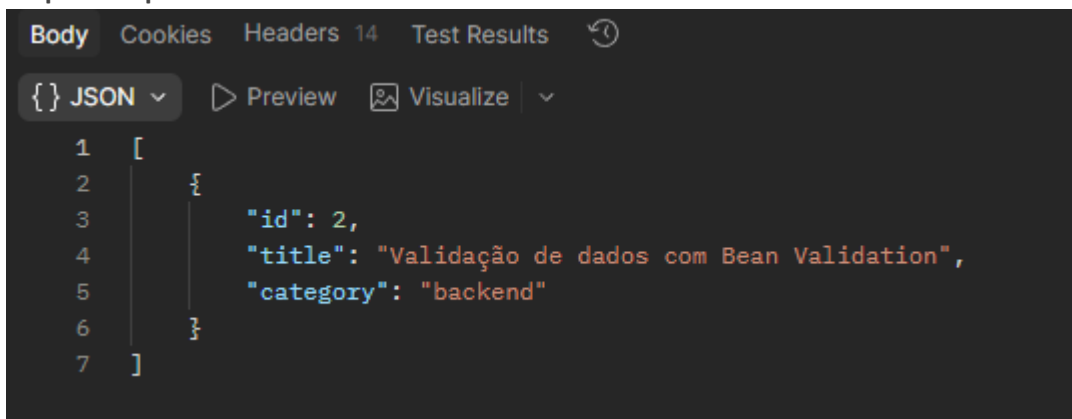
- Bearer Token

Exemplo de requisição



Essa requisição não possui body. As tags são enviadas diretamente na URL.

Resposta esperada



4.9 EXCLUSÃO DE TAG PERSONALIZADA

Objetivo

Permitir que o usuário remova uma tag personalizada previamente criada para um conteúdo.

Endpoint

- DELETE /content/{contentId}/personal-tags/{tagId}

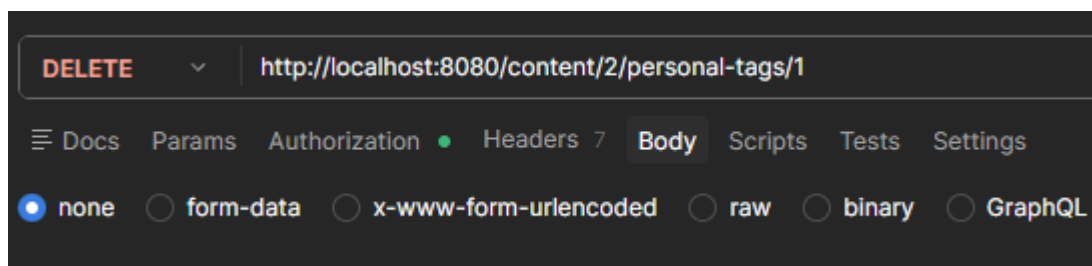
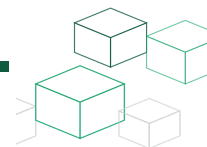
Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdo já cadastrado no sistema.
- Tag personalizada previamente criada

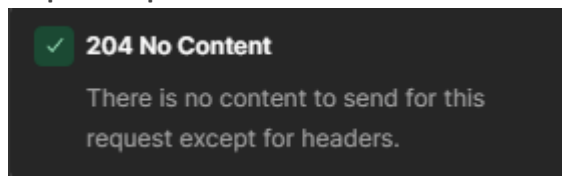
Autenticação

- Bearer Token

Exemplo de requisição



Resposta esperada



A resposta 204 No Content indica que a tag personalizada foi removida com sucesso e que não há corpo de resposta.

O que acontece internamente

1. O usuário autenticado seleciona um conteúdo da biblioteca.
2. O backend recebe a tag personalizada enviada pelo usuário.
3. O nome da tag é normalizado para evitar duplicidades.
4. A tag é salva na tabela `user_content_tag`.
5. A tag fica associada ao usuário e ao conteúdo.
6. O usuário pode listar, buscar ou remover suas próprias tags personalizadas.
7. As tags personalizadas não alteram as tags automáticas geradas pelo sistema.

Tabelas envolvidas

Tabela	Função
users	Armazena os usuários cadastrados no sistema.
content	Armazena os conteúdos técnicos cadastrados.
user_content_tag	Armazena as tags personalizadas criadas por cada usuário para cada conteúdo.

4.10 BUSCA SEMÂNTICA

Objetivo

Permitir que um usuário autenticado encontre conteúdos tecnicamente relacionados a partir de uma pesquisa por linguagem natural.

Endpoint

GET /content/search-similar?q=value

O parâmetro “q” representa o texto utilizado na pesquisa.

Esta requisição não possui body.

Exemplo

GET /content/semantic-search?query=Como proteger uma API REST com JWT

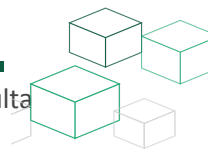
Autenticação

Bearer Token

Pré-requisitos

- Usuário autenticado.
- Conteúdos previamente indexados com embeddings.

Resposta esperada



A API retorna uma lista de conteúdos ordenados por similaridade semântica em relação à consulta realizada.

Cada item da resposta contém as informações do conteúdo e sua respectiva relevância para a pesquisa. Caso nenhum conteúdo semelhante seja encontrado, será retornada uma lista vazia.

```
{JSON} Preview Visualize
1 [
2   {
3     "id": 1022,
4     "title": "Segurança da Informação",
5     "content": "A segurança da informação é um dos pilares da tecnologia moderna. Aprender sobre segurança envolve compreender conceitos como criptografia, autenticação multifatorial, firewalls e políticas de acesso. Esse conhecimento é essencial para proteger dados pessoais e corporativos contra ataques cibernéticos. Além das ferramentas, é necessário desenvolver hábitos seguros, como criar senhas fortes, evitar redes públicas sem proteção e desconfiar de links suspeitos. O aprendizado nessa área prepara profissionais para atuar em setores críticos, como bancos, hospitais e órgãos governamentais. A segurança digital não é apenas uma questão técnica, mas também cultural, pois exige conscientização dos usuários e políticas organizacionais bem definidas.",
6     "category": "database",
7     "distance": 0.22396152288382908
8   },
9   {
10    "id": 1023
```

O que acontece internamente

1. O backend recebe o texto informado pelo usuário.
2. É gerado um embedding para a consulta utilizando o mesmo modelo empregado na indexação dos conteúdos.
3. O Oracle AI Vector Search compara esse vetor com os embeddings armazenados.
4. Os conteúdos são ordenados pela similaridade vetorial.
5. Os resultados mais relevantes são retornados ao usuário.

4.11 CONTEÚDOS RELACIONADOS

Objetivo

Permitir que o usuário visualize conteúdos relacionados ao conteúdo que está sendo acessado.

A recomendação é baseada na similaridade semântica entre os conteúdos, utilizando uma representação vetorial de cada documento.

Endpoint

GET /content/{id}/related

Onde **id** corresponde ao identificador do conteúdo utilizado como referência.

Esta requisição não possui body.

Exemplo

GET /content/15/recommendations

Autenticação

Bearer Token

Pré-requisitos

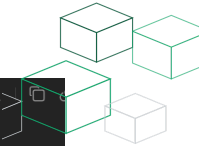
- Usuário autenticado.
- Conteúdo existente.
- Conteúdos previamente indexados com embeddings.

Resposta esperada

A API retorna uma lista de conteúdos semanticamente semelhantes ao conteúdo informado.

O próprio conteúdo utilizado como referência não é incluído na resposta.

Caso não existam recomendações, será retornada uma lista vazia.



```
{ } JSON Preview Visualize
1 [
2   {
3     "id": 1020,
4     "title": "Redes de Computadores e Comunicação",
5     "content": "As redes de computadores são responsáveis por conectar dispositivos e permitir a troca de informações. Aprender sobre redes envolve compreender protocolos como TCP/IP, DNS e HTTP, além de conceitos como endereçamento IP, roteamento e segurança. Esse conhecimento é fundamental para entender como a internet funciona e como os dados trafegam entre servidores e usuários. Estudantes que dominam redes conseguem configurar ambientes corporativos, otimizar conexões e identificar falhas de comunicação. Além disso, redes são a base para tecnologias emergentes como a Internet das Coisas (IoT), que depende de conexões estáveis e seguras para funcionar. O aprendizado nessa área abre portas para carreiras em administração de sistemas, segurança digital e infraestrutura de TI.",
6     "category": "backend",
7     "distance": 0.12192496947417086
8   },
9   {
10    "id": 1023,
11    "title": "Inteligência Artificial e Aprendizado de Máquina",
12    "content": "A inteligência artificial é uma das áreas mais promissoras da tecnologia. O aprendizado começa com conceitos de machine learning, onde algoritmos são treinados para reconhecer padrões em dados. Técnicas como regressão, classificação e redes neurais são
```

O que acontece internamente

1. O backend identifica o conteúdo solicitado.
2. É obtida a representação vetorial do conteúdo a partir do seu **embedding centroid**, calculado pela média dos embeddings dos seus chunks.
3. O Oracle AI Vector Search compara esse vetor com os centroides dos demais conteúdos.
4. Os conteúdos são ordenados pela similaridade vetorial.
5. Os conteúdos mais relacionados são retornados ao usuário.

4.12 CORREÇÃO DE TAGS

Objetivo

Permitir que o usuário confirme que as tags sugeridas automaticamente pelo sistema estão corretas, sem precisar enviar uma nova lista de tags.

Esse fluxo é usado quando o conteúdo foi marcado como baixa confiança, mas o usuário entende que as tags atuais estão adequadas.

Endpoint

- **PATCH /content/{id}/tags/confirm**

Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdo previamente cadastrado.
- Conteúdo com tags associadas.

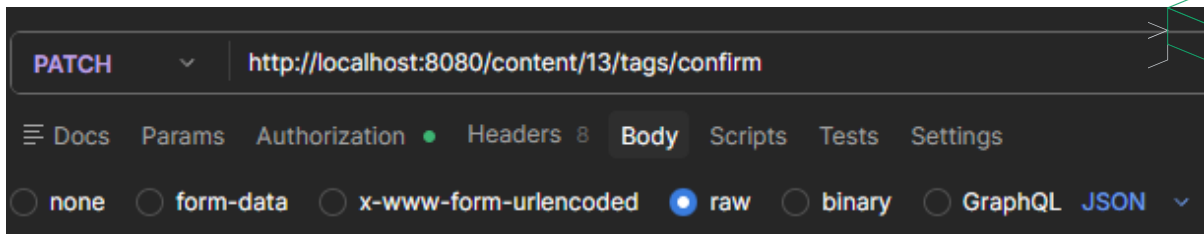
Autenticação

- Bearer Token

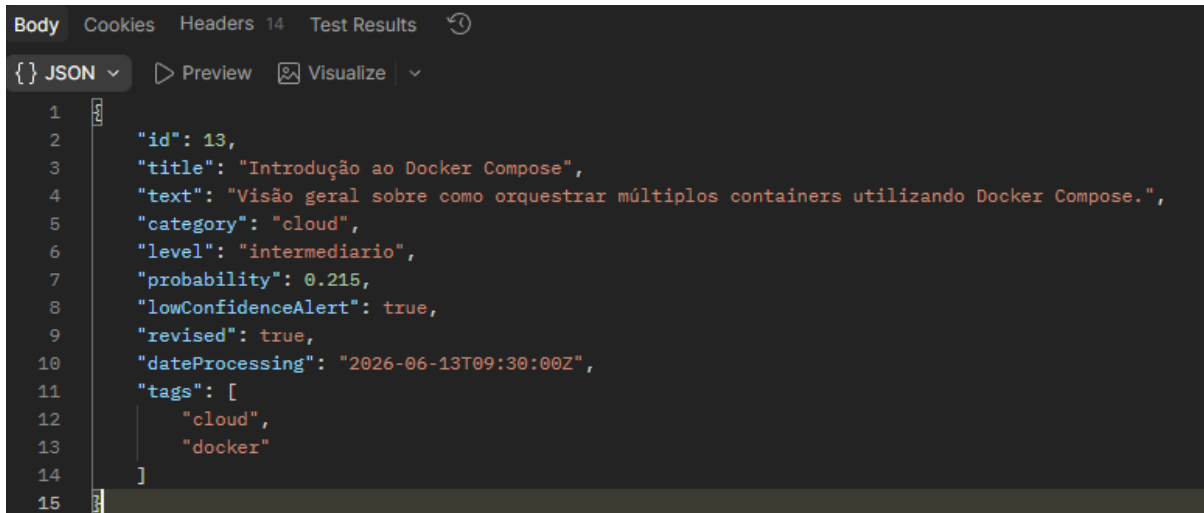
Body da requisição

Essa requisição não possui body. A confirmação é feita apenas pelo identificador do conteúdo enviado na URL.

Exemplo de requisição



Resposta esperada



A resposta retorna o conteúdo atualizado, mantendo as tags existentes e marcando o conteúdo como revisado.

O que acontece internamente

1. O backend recebe o identificador do conteúdo pela URL.
2. O sistema localiza o conteúdo no banco de dados.
3. As tags existentes são mantidas.
4. O conteúdo é marcado como revisado.
5. A API retorna o conteúdo atualizado.

4.13 CORREÇÃO DE TAGS INCORRETAS

Objetivo

Permitir que um usuário autenticado corrija as tags associadas a um conteúdo quando a classificação automática do sistema não estiver adequada.

Essa funcionalidade é usada quando o sistema sugere tags incorretas ou pouco relacionadas ao conteúdo. Nesse caso, o usuário envia uma nova lista de tags, que substitui as tags anteriores.

Endpoint

- **PUT /content/{id}/tags**

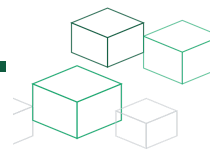
Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.
- Conteúdo previamente cadastrado.
- Conteúdo com tags associadas.

Autenticação

- Bearer Token

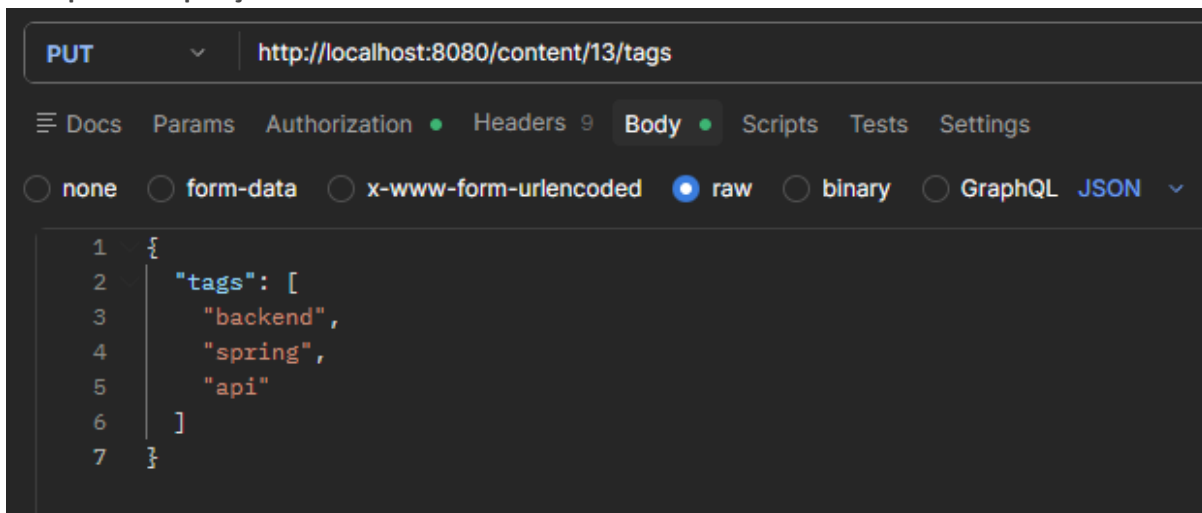
Body da requisição



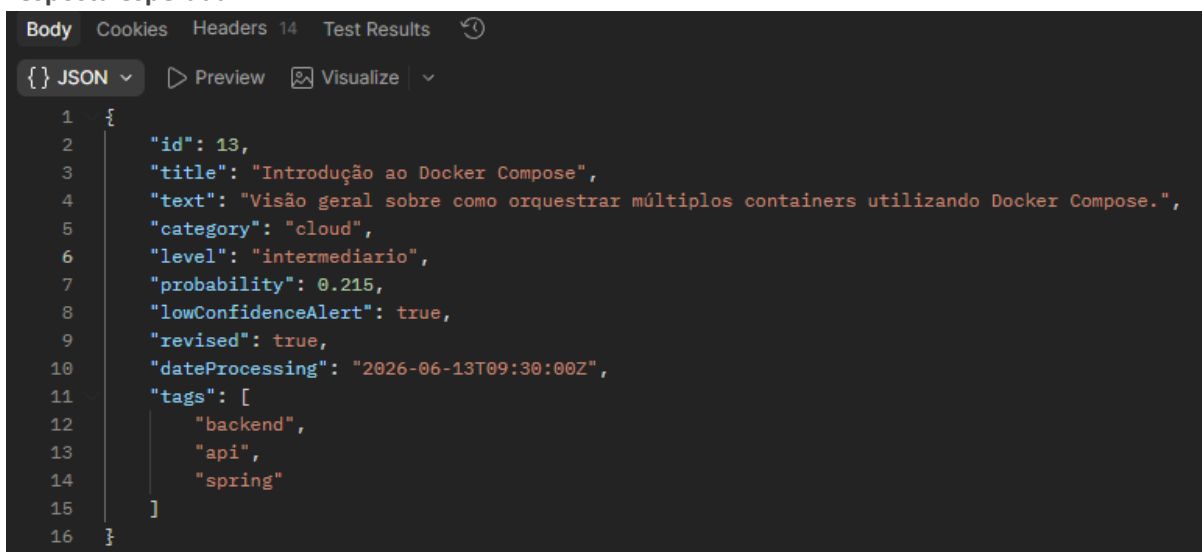
```
{
  "tags": [
    "backend",
    "spring",
    "api"
  ]
}
```

O campo tags recebe a nova lista de tags que deverá substituir as tags anteriores do conteúdo.

Exemplo de requisição



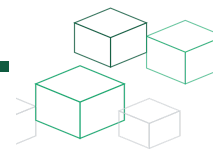
Resposta esperada



A resposta retorna o conteúdo atualizado, já com as novas tags enviadas pelo usuário. As tags anteriores são substituídas, e o campo revised passa a indicar que o conteúdo foi revisado.

O que acontece internamente

1. O backend recebe o ID do conteúdo pela URL.
2. O backend recebe a nova lista de tags pelo body da requisição.
3. O sistema localiza o conteúdo no banco de dados.
4. As tags antigas associadas ao conteúdo são removidas.
5. As novas tags são normalizadas e associadas ao conteúdo.
6. O conteúdo é marcado como revisado.
7. A API retorna o conteúdo atualizado



4.14 FAVORITOS DO USUÁRIO

Objetivo

Permite o usuário:

- Marcar conteúdos como favoritos.
- Desmarcar conteúdos que estão como favoritos.
- Listar todos os conteúdos marcados como favoritos.

Endpoints

- **POST/content/{id}/favorite**
- **DELETE/content/{id}/favorite**
- **GET/content/favorites**

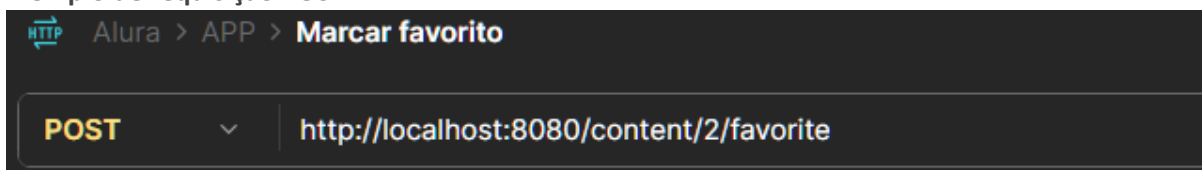
Pré-requisitos

- Backend em execução.
- Banco de dados conectado.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado.
- Conteúdos cadastrados no Banco de Dados.
- Conteúdo previamente cadastrado.

Autenticação

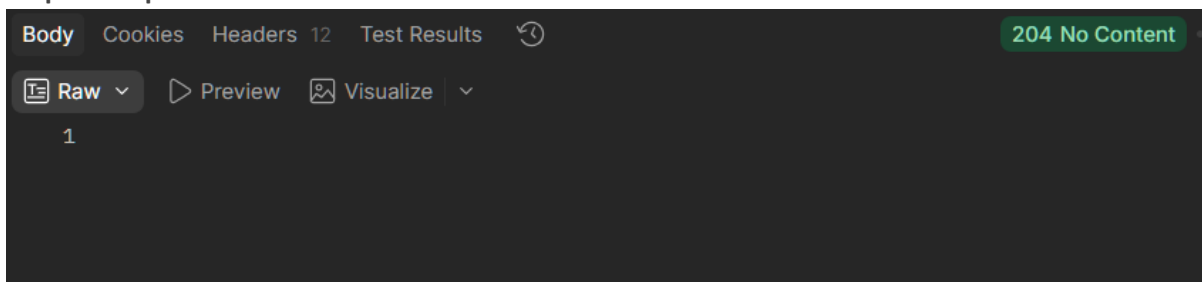
- Bearer Token

Exemplo de requisição POST



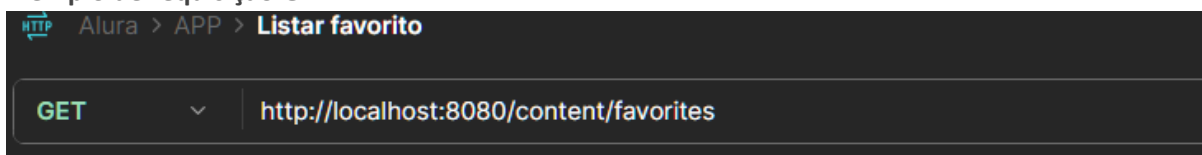
Aqui colocados o ID do conteúdo que desejamos favoritar, nesse caso é o 2

Resposta esperada do POST



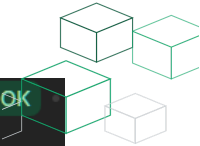
O código de status 204 confirma a marcação do conteúdo como favorito

Exemplo de requisição GET



Esse vai retornar todos os conteúdos que estão marcados como favoritos.

Resposta esperada do GET



```
Body Cookies Headers 14 Test Results 200 OK
{ } JSON Preview Visualize
1 [
2   {
3     "id": 2,
4     "title": "Validação de dados com Bean Validation",
5     "category": "backend",
6     "level": "intermediário"
7   }
8 ]
```

Exemplo de requisição DELETE

```
HTTP Alura > APP > Desmarcar favorito
DELETE http://localhost:8080/content/2/favorite
```

Aqui colocamos o ID do conteúdo que desejamos desmarcar como favorito.

Resposta esperada do DELETE

```
Body Cookies Headers 12 Test Results 204 No Content
Raw Preview Visualize
1
```

Como no POST, o DELETE não vai retornar um JSON mas vai apenas atualizar o conteúdo.

O que acontece internamente

1. O Backend vai receber o ID do conteúdo pela URL.
2. O ID recebido vai ser verificado pelo Banco de Dados para ver se o conteúdo existe.
3. Após a verificação, esse conteúdo vai ser marcado como favorito.
4. Agora o usuário pode Desmarcar esse conteúdo favoritado ou Listar todos os conteúdos favoritados por ele.

4.15 SWAGGER

Objetivo

Permite o usuário a acessar uma documentação sobre a API, mostrando ao usuário, todos os endpoints da API.

URL

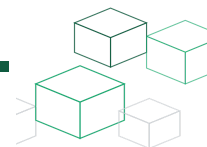
- <http://localhost:8080/swagger-ui/index.html>

Pré-requisitos

- Backend em execução.

Autenticação

- Não precisa para poder utilizar o Swagger, só vai usar uma autenticação nos endpoints.



4.16 CLASSIFICAÇÃO DE NÍVEL DE CONTEÚDO

Objetivo

Além de categorizar o conteúdo e gerar tags automaticamente, o serviço de classificação também identifica o nível de dificuldade do conteúdo técnico enviado (por exemplo, iniciante, intermediário ou avançado), ajudando o usuário a filtrar e entender rapidamente a complexidade do material.

Endpoint

- POST /content

Pré-requisitos

- Backend em execução.
- Banco de dados Oracle disponível.
- Usuário cadastrado.
- Login realizado.
- Token JWT válido.
- Header Authorization configurado no Postman.

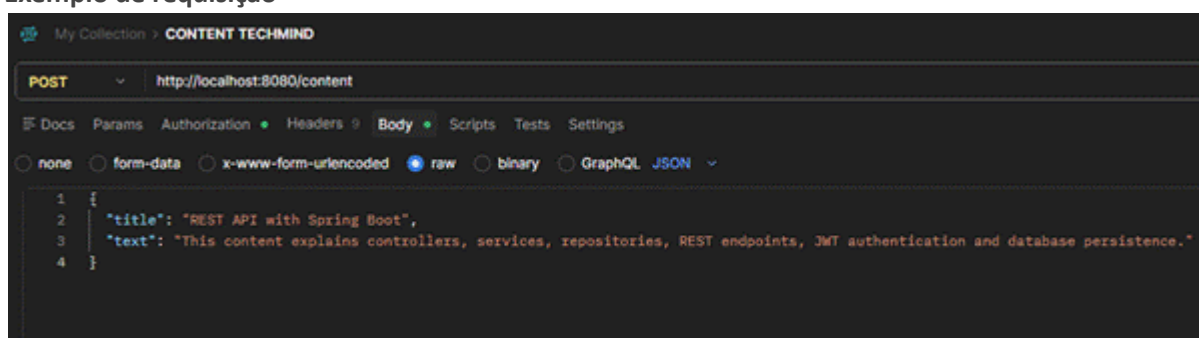
Autenticação

- Bearer Token

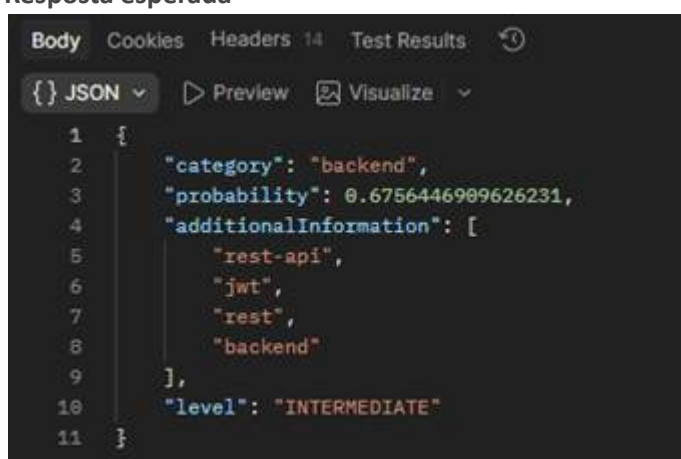
Body da requisição

```
{
  "title": "value",
  "text": "value"
}
```

Exemplo de requisição



Resposta esperada



A resposta retorna, além da categoria e das tags já documentadas anteriormente, o campo level, que indica o nível de dificuldade classificado automaticamente para o conteúdo (no exemplo, "INTERMEDIATE").

O que acontece internamente

1. O backend recebe título e texto.
2. O conteúdo passa pelo serviço de classificação.
3. A categoria, as tags e o nível de dificuldade são gerados na mesma etapa de processamento.



-
4. O conteúdo é salvo no banco de dados, já com o nível associado.
 5. O conteúdo fica disponível para busca e consulta, podendo futuramente ser filtrado também por nível.